

Building Your First Linked List in Python

Mastering core data structures through the Train & Bogie analogy

1. Introduction to Linked Lists

In Python, we often store data in lists. However, under the hood, standard contiguous lists and **Linked Lists** organize memory very differently. A Linked List is a linear data structure where elements are not stored in adjacent memory locations. Instead, each element points to the next one, forming a dynamic chain.



The Train Analogy (Vande Bharat Express)

Think of a Linked List as a train!

- **Engine:** Head pointer (starts the whole chain).
- **Bogie (Coach):** Node (contains data + hook to next coach).
- **Coupling Hook:** Next pointer.
- **Last Bogie Hook:** Points to None (end of the track).

2. Anatomy of a Node (The Bogie)

Every single item in a linked list is called a **Node**. Each Node consists of two fundamental parts:

Component	Python Attribute	Train Analogy
Data Box	<code>self.data</code>	Carries passengers or luggage inside the bogie.
Next Pointer	<code>self.next</code>	The coupling hook connecting to the next bogie.

Here is how we define the Node class in Python:

```
class Node:
    def __init__(self, data):
        self.data = data    # Stores what's inside the bogie
        self.next = None   # No bogie hooked yet!
```

Connecting Two Bogies Manually:

```
bogie1 = Node("Passengers")
bogie2 = Node("Pantry")

# Link bogie1 to bogie2 using the next pointer
bogie1.next = bogie2
```

3. The LinkedList Class (The Train Engine)

To manage our nodes, we wrap them inside a `LinkedList` class. The class maintains a reference to the very first node, known as the **head**.

```
class LinkedList:
    def __init__(self):
        self.head = None    # Engine initialized without any bogies attached
```

 **Key Rule:** `head = None` means the train is currently empty. The head is where every journey begins—without it, we lose reference to the entire chain!

4. Core Operations: Append & Display

1. Adding Bogies (The append Method):

To add elements to the end of our list, we follow a four-step process:

1. **Create Node:** Instantiate `new_node = Node(data)`.
2. **Check Head:** If `self.head` is `None`, assign `self.head = new_node`.
3. **Traverse:** Walk from the head to the last bogie using a loop: `while last.next: last = last.next`.
4. **Attach:** Hook the new node by setting `last.next = new_node`.

2. The Ticket Collector (The display Method):

To view our list, we traverse from head to `None`, printing each node's data along the way.

```
def display(self):
    current = self.head
    while current:
        print(f"[{current.data}] -> ", end="")
        current = current.next
    print("None")
```

5. Hands-On Challenge: Build the "Snack Express"

Put your knowledge into practice! Create a `LinkedList` instance and attach your favorite snacks as bogies.

```
# Build the Snack Express train!
samosa_express = LinkedList()

samosa_express.append("Samosa")
samosa_express.append("Jalebi")
samosa_express.append("Dhokla")

# Display the full train in console
samosa_express.display()
```

Expected Output:

```
[Samosa] -> [Jalebi] -> [Dhokla] -> None
```

Student Practice Questions

Test your understanding of Python Linked Lists & Node traversal

Complete the following practice questions to reinforce your understanding. Write down your explanations and code snippets on paper or in your code editor.

Question 1: Multiple Choice

In a standard Python Node class implementation, what does the attribute `self.next` store?

- (a) The integer index of the next node
- (b) A reference/pointer to the next Node object
- (c) The raw data value of the next bogie
- (d) The total count of all nodes in the list

Question 2: Conceptual Understanding

What is the value of `self.next` for the last bogie (tail node) in a singly linked list?

- (a) `0`
- (b) `self.head`
- (c) `None`
- (d) `-1`

Question 3: Code Analysis & Bug Hunt

Consider the following faulty implementation of the append method. What will happen if you attempt to call `append("Chai")` on an empty list (where `self.head` is `None`)? Identify the exact error.

```
def append(self, data):
    new_node = Node(data)
    current = self.head
    while current.next:
        current = current.next
    current.next = new_node
```

Question 4: Short Answer & Logic

If Coach B1 only knows which coach directly follows it, explain how the train Engine (Head) is able to access or reach Coach B5 at the end of the train.

Question 5: Coding Challenge — Prepend Operation

Write a Python method named `prepend(self, data)` for the `LinkedList` class that inserts a new bogie at the **very beginning** of the train (making it the new head). *Hint: Ensure you don't lose the existing head!*

Question 6: Coding Challenge — Length Counter

Write a Python method named `get_length(self)` that traverses the linked list and returns the total number of bogies (nodes) present in the train.